

Extracting and Classifying Requirements from Software Engineering Contracts

Abhishek Sainani
TCS Research
Pune, India
a.sainani@tcs.com

Preethu Rose Anish
TCS Research
Pune, India
preethu.rose@tcs.com

Vivek Joshi
TCS Research
Pune, India
vivek.joshi3@tcs.com

Smita Ghaisas
TCS Research
Pune, India
smita.ghaisas@tcs.com

Abstract—In this paper, we present our work on extracting and classifying requirements from large software engineering contracts. Typically, the process of requirements elicitation begins after a contractual agreement is signed by all participants. Our interactions with the legal compliance team in a large vendor organization reveal that business contracts can help in the identification of high-level requirements relevant to the success of software engineering projects. We posit that requirements engineering as a discipline has an even wider scope than software engineering of which it is traditionally considered to be a sub-discipline. This is because software engineering-specific requirements are but a part of the success story of any large project. The requirements that emerge from contracts are obligatory in nature, whether or not they pertain to core software development. Therefore, it is important that these are extracted and classified for the benefit of software engineers and other stakeholders responsible for a project. We discuss the results of an exploratory study and a range of experiments from the use of regular expressions to Bidirectional Encoder Representations from Transformers for automating the extraction and classification of requirements from software engineering contracts. With Bidirectional Encoder Representations from Transformers, we obtained a high f-score of greater than eighty four percent for classification of requirements.

Index Terms—Software engineering contracts, requirements, deep learning, extraction, classification

I. INTRODUCTION

Information Technology Outsourcing (ITO) has become the predominant mode of acquiring information systems services over the last two decades [1]. The degree of subcontracting in ITO varies across the whole spectrum, ranging from just a part of the IT function to outsourcing of an entire Software Development Life Cycle (SDLC) of a project [2]. Contracts are vital components of ITO. According to [4], 60-80% of business transactions are governed by contractual agreements and a typical Fortune 1000 Company maintains 20,000 to 40,000 active contracts at any given time.

Contracts are legally binding descriptions of business service engagements. Any large organization must unavoidably invest significant resources in developing and concluding contracts, as the contracts it enters define its legal relations with participants [5]. Contracts cover a wide variety of topics including reward

and penalty, service level agreements, time frames, periodic reviews and exit strategies. In this paper, we explore contracts as a possible source of requirements.

Typically, the process of requirements elicitation begins after a contractual agreement is signed by all participants. Requirements elicitation includes inputs from clients, existing systems and documentation, users (existing and potential), standards, legislations and domain experts. Our interactions with the legal compliance team of a large software vendor however revealed that business contracts can help in the identification of high-level requirements and at times, may even point to an architectural vision. For example, the following statement in a contracts document implies architectural requirements such as network security, encryption of data and other security procedures.

“Vendor shall maintain reasonable operating standards and security procedures, and shall use commercially reasonable efforts to secure client’s Personal Data and Confidential Information through the use of appropriate physical and logical security measures including, but not limited to, appropriate network security and encryption technologies.”

This observation motivated us to conduct an exploratory study to find out if contracts can indeed be a useful source of requirements knowledge and if so, what kind of requirements exist in contracts. We formulated the following Research Questions (RQs) to anchor our study.

RQ1. Can contracts be leveraged as a useful source of requirements knowledge even before the traditional Requirements Engineering (RE) exercise begins?

RQ2. What are the different kinds of requirements that can be gleaned from contractual obligations?

Our study (discussed in section II) reveals that ~60% of the requirements gleaned from contracts are governance-specific while the remaining ~40% are architecture-specific. The requirements that emerge from contracts are obligatory in nature, whether or not they pertain to core software development. Therefore, it is important that these are identified early on for the benefit of software engineers and other stakeholders responsible for a project.

However, mining any kind of information from business contracts is a challenging task. Contracts are large documents with convoluted Legalese-like complex language [34] and ambiguities, unlike tweets and product reviews, which have been

the traditional datasets for Information Retrieval (IR) techniques [33]. We therefore automate the extraction and classification of requirements from large contract documents. We performed a range of experiments for the automation, starting from use of regular expressions to Bidirectional Encoder Representations from Transformers (BERT), and achieved a high F-score of 87.12%. To the best of our knowledge, this is the first attempt to leverage contracts as a dataset for extracting and classifying requirements pertinent to Software Engineering (SE) projects.

The contribution of this paper is two-fold: first, we identify requirement types present in SE contracts through an exploratory study, and second, we automate the extraction and classification of the requirements from the contracts. The automation study presented is a part of an initiative for AI-powered compliance with legal documents such as regulations and contracts.

The rest of the paper is organized as follows: In Section II, we explain the exploratory study of contract documents. In Section III, we explain our experiments to automate the extraction and classification of requirement types from contracts. Section IV is on related work. Section V presents a validation study. Section VI discusses threats to validity and Section VII concludes the paper.

II. EXPLORATORY STUDY OF CONTRACT DOCUMENTS

The obligations outlined in contracts are written with a purpose to ensure compliance from all stakeholders. Since Information Technology (IT) is the backbone of all modern businesses, we expected the clauses in contracts to contain several details pertinent to SE requirements, in addition to governance related obligations. We studied contracts documents with an objective to answer RQ1 and RQ2.

Our dataset for the exploratory study consists of 20 expired SE contracts from 9 application domains, which are, healthcare, automotive, finance, banking, pharmaceuticals, telecom, technology, clothing-retail and supermarket. All the contracts are prepared by legal experts of the vendor organization and of the 20 multinational customers who have their business activities spread across the globe. The size of each contract document ranges from 100 to 500 pages. A total of 18,614 sentences are extracted from these contracts. The number of words in each sentence ranges from 20 to 250.

A. Exploratory Study Method

To identify various kinds of requirements present in a contract document, we followed the guidelines of Charmaz’ Constructive Grounded Theory approach [32]. One author (who is a software engineer and a researcher) of this paper, two software engineers from a business unit, along with two members of the contracts governance team participated in this analysis. The software engineers had experience in working on software development projects in different domains and had a professional experience of more than seven years. The researcher has also previously worked with large SE projects. The contracts governance team comprised of two senior professionals with more than fifteen years of experience. Henceforth, we will refer to all five of them as participants. Each participant first individually analyzed each sentence in the contract documents and attached a coding

word to a portion of the text – a phrase or a sentence. The coding words selected by the software engineers were relevant to the Information Technology (IT) and SE-specific requirements whereas the coding words selected by the contracts governance team were relevant to the governance or operational aspects of the contracts. Each participant took an average of 1 minute to understand a sentence and assign a coding word to it. Therefore, to analyze our dataset that consists of 18,614 sentences, it took about 39 person-working days or 8 working days for the five participants. For the next 8 working days, the software engineers and contracts governance team worked together to study the various coding words to arrive at higher-level categories. The participants also shared and clarified assumptions to arrive at a consensus regarding how and why particular concepts were important. Finally, 2 senior legal experts (more than 10 years of experience) and 1 software engineer with more than 15 years of experience independently reviewed the concepts. This review did not result in any additions or modifications to the concepts.

B. Exploratory Study Results

Our study revealed the following insights into the nature of information in contract documents:

1. A contract document consists of two types of clauses, obligations and non-obligations. We examined the obligation statements in the contracts as potential sources of requirements knowledge. The non-obligation statements are “information”, “definition” or “fact” statements that do not translate to software requirements.

2. The obligations in a contract document could be a credible source of different kinds of requirements that may be IT-specific (for example, security or privacy related). Some might be governance-specific and outside the scope of core software development and yet important and mandatory for delivering a successful software project. They may even be architecturally significant and may need an early attention in the life cycle of a SE project.

3. 14 different kinds of requirements were identified from the obligation sentences. We refer to them as requirement types. Table I presents the distribution of sentences of each requirement type in our dataset.

TABLE I: DISTRIBUTION OF REQUIREMENTS

Contract Requirement Types	No. of Instances
Project Delivery	2005
Information Security	1577
Legal Process	650
Screening/Onboarding	196
Data Privacy	365
Vendor Corporate	39
Improvement and Innovation	22
Personnel Allocation	162
HR Client Policy	163
HR Laws	83
Third Party IP Licensed	62
Vendor IP Licensed	54
Export Laws	58
Standards	36

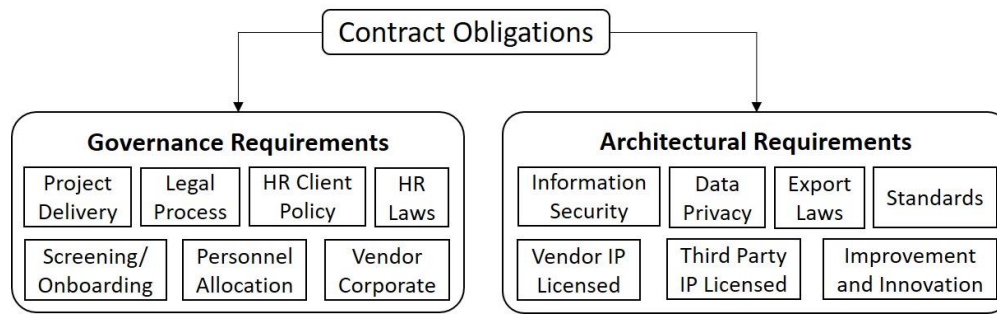


Figure 1. Requirement Types in Contract Documents

Fig. 1 pictorially shows the classification of requirements in contract documents. As indicated in fig. 1, *Information Security*, *Data Privacy*, *Improvement and Innovation*, *Vendor IP Licensed*, *Third Party IP Licensed*, *Export Laws*, *Standards* are SE-specific (particularly, significant for architecture and therefore named as “Architectural” requirements). The remaining requirement types: i.e. *Project Delivery*, *Legal Process*, *Screening/Onboarding*, *Vendor Corporate*, *HR Client Policy*, *HR Laws* and *Personnel Allocation* are relevant to the governance aspect of the project.

4. We also observed that an obligation statement could belong to one or more requirement types. For instance, the following clause contains audit information about *Data Privacy* as well as *Information Security*.

“Without limiting any of its other obligations under this Agreement, the Supplier shall ensure that if the <<client name>> data is placed on a portable electronic device (including laptops, memory sticks and back-up tapes) or transmitted electronically, it is securely encrypted in accordance with Good Industry Practice.”

Point 2 answers RQ1 and the points 3 and 4 provides a detailed answer to RQ2.

Next, we provide a definition and explanation of each of the requirement types along with examples. All the examples are obfuscated for confidentiality reasons. We first explain architectural requirement types and then discuss the governance-specific requirement types.

Architectural Requirements:

Requirements that influence the architecture of the software system are called architectural requirements. We found the following requirement types in business contracts that are architecturally important: *Information Security*, *Data Privacy*, *Improvement and Innovation*, *Vendor IP Licensed*, *Third Party IP Licensed*, *Export Laws*, *Standards*.

We explain each in detail:

Information Security: This deals with access to information, retention and destruction of data, procedure to follow during a security incident and maintaining business continuity and disaster recovery plan. An example contractual obligation is:

“The vendor undertakes that it has and shall continue to have in place and keep up to date business continuity plans and disaster recovery plans sufficient to minimize the possibility of any interruption in the services and allow the rapid restoration of the services should they, nonetheless, be disrupted by a disaster.”

Business continuity plans and disaster recovery plans are required to keep the customer in business, with a minimum of disruption, in the event of a major disaster. The network could have an outage, a latest application push might introduce a critical bug, or there might be a natural disaster. For a high-availability application, distribution between two physically separated data centers, replication management, and more is required. This indicates the need to include the requirement for high availability in the architectural design.

Data Privacy: This deals with various Data Privacy laws and requirements to adhere to Data Privacy policies of client. An example contractual obligation is:

“Vendors are required to protect the personal and confidential information they receive about client and shall not transfer, store, access or otherwise process such information provided by client without client’s prior consent.”

Data privacy requirement advocates architectural consideration to restrict access to data through application-level controls, use of security architectures to avoid creating a single point of trust in the systems, explore federated architectures that let users retrieve and view data without compromising data security and maintaining and analyzing audit logs.

Improvement and Innovation: This deals with improvements promised in the contract, such as improvements in tools, techniques, innovation etc. An example contractual obligation is:

“Vendor will, on a continuous basis, (i) identify ways to improve the service levels, and (ii) identify and apply proven techniques and tools from other installations within its operations that would benefit client either operationally or financially.”

Large organizations often have a repository of reusable techniques, components and tools that have proven to be successful in past projects. As indicated in [20], such techniques, components and tool, if needed to be integrated with the core application, will need architectural considerations.

Vendor IP Licensed: This deals with procuring or using license of a vendor Intellectual Property (IP) in the software to be developed.

An example contractual obligation is:

“The parties acknowledge that, during the course of providing services, vendor may use its own pre-existing proprietary materials, software and tools or components developed or owned by vendor prior to or independent of the services performed

hereunder or any enhancements or improvements thereof made as part of the services hereunder.”

This obligation deals with usage of vendor’s IP that may include software, tools, or components that needs to be integrated with the core solution and hence affect the architecture of the software system.

Third Party IP Licensed – This deals with procuring or using license of a third party IP in the software to be developed. An example contractual obligation is:

“The parties acknowledge that, during the course of providing services, vendor may use third party licensed materials, software and tools or components to develop the services performed hereunder or any enhancements or improvements thereof made as part of the services hereunder.”

This deals with usage of third party components and therefore has significant architectural impact [20].

Export Laws: These deal with export laws and sanctions to be followed when transferring data, technology or hardware outside the country. An example contractual obligation is:

“Each party agrees that it will not use, distribute, transfer, or transmit any products, software, data, or technical information (even if incorporated into other products) in violation of applicable import or export laws or regulations.”

Data / technology / hardware transfer refers to the secure exchange of large files or hardware between systems or organizations. Because the data is moving beyond the enterprise perimeter, challenges such as security and complexity needs to be addressed and they require careful architectural considerations.

Standards: This deals with various audit and industry standards that needs to be complied with. An example contractual obligation is:

“Vendor shall comply with all applicable industry standards of ISO 9000/1, ISO 20000, and ISO 22301 concerning privacy, data protection, confidentiality or information security.”

Standards are subject to audits and audits are established to be an architecturally significant requirement [20].

Next, we explain Governance related requirements.

Governance Requirements:

Requirements that deal with the operational aspects of the business are called Governance Requirements. We found the following requirement types in business contracts that are governance-specific: *Project Delivery*, *Legal Process*, *Screening/Onboarding*, *Vendor Corporate*, *HR Client Policy*, *HR Laws* and *Personnel Allocation*.

Next, we explain each of these in detail:

Project Delivery: This deals with Service Level Agreements (SLAs), acceptance process, fixing defective deliverables, warranty, subcontracting and termination of the contract. An example contractual obligation is:

“The Vendor shall obtain the prior written agreement of the client in the form of a change order for any change in the manner in which the Vendor performs and/or delivers the services, including any change that involves any change in the scope of the

services including any change to a Vendor services locations or in the services elements performed at or from any of the Vendor services locations.”

Legal Process: This deals with dispute resolution, insurance obligations, benchmarking, step-in and breach. An example obligation is:

“If any dispute arises out of this agreement, which is not resolved in accordance with the preceding sub-clauses of this clause within 21 days of its referral, the Parties will attempt to settle it by mediation in accordance with the model procedure of the Centre for Effective Dispute Resolution.”

Screening/Onboarding: This deal with various background-screening obligations required for new employees or transitioned employees. An example obligation is:

“Prior to performing services for client and to the extent permitted by applicable law, Supplier shall conduct background investigations and drug screening of all Supplier Personnel who are to be assigned to perform services for client in accordance with highest industry standards and best practices where services will be performed.”

Vendor Corporate: This deal with ownership changes and financial health of vendor. An example obligation is:

“If any Client believes that such an event or circumstance has arisen, the Vendor will, to the extent permitted by Law, provide all such information as is reasonable to address the Client’s concerns around the financial standing of the Vendor or the relevant member of the Vendor group.”

HR Client Policy: This deals with solicitation of employees of the other party, and adhering to client policies related to Human Resource (HR). An example obligation is:

“Neither Party shall solicit for employment or knowingly hire any personnel of the other Party who has had a direct involvement in any Work Order or Statement of Work, without such other Party’s express written consent.”

HR Laws: This deals with adherence to HR related laws. An example obligation is:

“Vendor must comply with all applicable immigration laws and regulations of the countries in which its Personnel perform any service for client.”

Personnel Allocation: This deals with allocation of personnel to a project, assignment of roles and responsibilities in the project, and removal of personnel from a project. An example obligation is:

“If Client determines that the performance of Vendor’s management team is unsatisfactory, Vendor shall correct such performance or provide a replacement acceptable to Client within five (5) months of receipt of a written notice from Client of such determination.”

In the next section, we present details of our solution design to automate the extraction and classification of the obligations along the requirement types identified in the exploratory study.

III. AUTOMATING EXTRACTION AND CLASSIFICATION OF REQUIREMENTS

As described earlier, the manual extraction and classification of requirements from a set of only 20 contracts took 39 person-working days. Given that a large organization has to manage thousands of contractual obligations, it is easy to appreciate the value of automation. To automate the extraction and classification of requirement types present in contract documents, we designed a solution approach as depicted in fig. 2. For our experiments, the “ground truth” was the labeled dataset created as a result of our exploratory study detailed in Section II.

Many organizations still follow a traditional software delivery method wherein signed contracts are sent by postal mail or fax, scanned copies are emailed or are personally handed over by a representative of one of the parties involved in the contract [6]. For attempting any kind of automation on these contracts, the key contracts metadata and text from each contract document must be made amenable for search and querying. We therefore used an in-house tool that employs a licensed Optical Character Recognition (OCR) application to convert the scanned contracts into an editable electronic format. We refer to the resulting contracts as digitized contracts. The digitized contracts are in an HTML format. The HTML tags from the digitized contracts were used to identify the sections and the contractual clauses within the sections. The in-house legal team then reviewed these, and errors, if any, were manually corrected. In the third step, (1) the sentences were identified and extracted from the clauses using a sentence tokenizer, and (2) a binary classifier was used to classify the extracted contractual sentences as an obligation or a non-obligation. The obligations are, potentially, requirements. These potential requirements are then classified into various requirement types discovered in the exploratory study.

A. Extracting Obligations as Potential Requirements from Contracts

In this section, we describe (1) the techniques used for extraction of obligation sentences as potential requirements from contracts, (2) the results obtained using each technique, and (3) the discussion on the results. Extracting obligation as potential requirements is essentially a binary classification task that labels each sentence in the contracts dataset as an “obligation” or a “non-obligation”. Our dataset consists of 5472 obligation sentences and 13,142 non-obligation sentences. For the binary classification task, we applied traditional machine learning approaches such as Naïve Bayes, Random Forest and Support Vector Machine, and a deep learning architecture called Bidirectional Long Short-Term Memory (BiLSTM). For all the techniques, we have used nested cross validation. In the outer loop,

we used grid search method for optimizing hyper parameter values and in the inner loop, we used stratified five-fold cross-validation of the dataset for training a model on a given set of hyper parameter values obtained from the outer loop.

1) Traditional Machine Learning Techniques

In this subsection, we discuss in detail our experiment with three different machine-learning approaches – Naïve Bayes, Random Forest and Support Vector Machine (SVM). We conducted our experiments using the implementation of machine learning techniques in Scikit-learn module in Python [38] and implementation of BiLSTM in Keras library in Python [39].

In machine learning based solution for binary classification task, the two steps to create the model for the required classification task are: dataset preparation and training the learning model. We next explain each of these steps in detail.

Dataset Preparation: This step is divided as follows:

- i) *Preprocessing:* This includes removing special characters and stop words, converting the sentences to lowercase, and lemmatization.
- ii) *Feature selection:* This involves automatically identifying useful bigrams and trigrams for each sentence based on their frequency in the dataset and adding them to the corresponding sentence in the dataset. We obtained best results with adding bigrams and trigrams occurring 3 or more times in the dataset.
- iii) *Feature encoding:* This involves transforming the dataset features into a Term Frequency – Inverse Document Frequency (TF-IDF) matrix.

Training the Learning Model: This step takes the TF-IDF matrix representation of sentences in the dataset as input to a machine learning technique for training the model. Table II presents various hyper parameter values that gives best results on our dataset for the machine learning techniques we used in our experiments. Table III provides the definition of these hyper parameters. To handle the imbalance in the dataset, for Random Forest and SVM we use the hyper parameter ‘class_weight’ and for Naïve Bayes, we use its variant called ‘Complement Naïve Bayes’.

TABLE II. HYPER PARAMETER VALUES

Machine Learning Technique	Hyper parameter values
Complement Naïve Bayes	alpha = 0.01
Support Vector Machines	kernel='rbf', gamma='scale', class_weight='balanced', C=10
Random Forests	n_estimators=200, criterion= 'gini', class_weight='balanced'

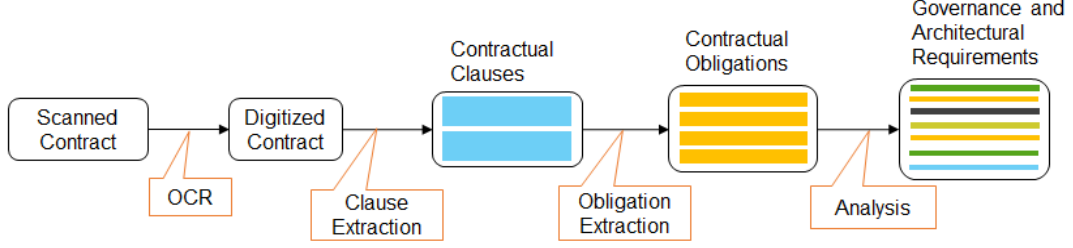


Figure 2. Solution Approach to Automate the Extraction and Classification of Requirement Types

TABLE III. HYPER PARAMETER DEFINITION

Hyper parameter	Definition
Alpha	Additive smoothing parameter
Kernel	Kernel type used
Gamma	Kernel coefficient
C	Regularization parameter
n_estimators	Number of trees in the forest
Criterion	The function to measure the quality of the split.
class_weight	To automatically adjust weights inversely proportional to class frequencies.

2) BiLSTM Architecture

In this subsection, we discuss the BiLSTM architecture for the binary classification task. This architecture consists of a word-embedding layer, a BiLSTM layer, an attention layer and an output layer as shown in fig. 3.

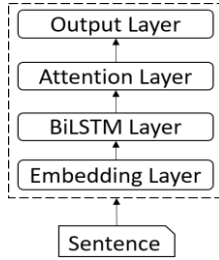


Figure 3. BiLSTM Architecture

Next, we briefly describe the role of each layer:

Embedding Layer: The words in the obligation sentences are converted to a vector form using a word embedding technique [8]. We use Word2Vec model (Skipgram with negative sampling [9]) as the word embedding technique to initialize the embedding layer. This Word2Vec model is trained only on our contracts dataset. We also experimented with Word2Vec model trained only on Wikipedia and with Word2Vec model first trained on Wikipedia and then on our dataset. However, the Word2Vec model trained only on our contracts dataset provided us with the best results.

BiLSTM Layer: The vector representation of each word in the sentence is provided as input to a BiLSTM [1] model to learn the long-distance contextual dependency among words, in both forward and reverse direction. This enables the model to understand the complete context for a given word.

Attention Layer: We train a single-layer feed-forward neural network to learn the importance of each word in a given sentence within the context of the classes. Using this, we obtain the weights corresponding to each word based on the importance of the word in the sentence. Finally, we obtain the final score based on an aggregation over the weighted output of all the nodes.

Output Layer: We use sigmoid function in the output layer because it normalizes the final score by converting it to a value between 0 and 1. We obtained the best results with threshold value of 0.5 such that input having score higher than 0.5 is labeled as ‘obligation’ and as ‘non-obligation’ otherwise. Hence, the output layer has only one node.

We use binary cross-entropy loss or Sigmoid cross-entropy loss as the loss function in our architecture. Moreover, to address the class imbalance problem of our dataset, we have used the weighted form of this loss function, as defined below:

$$WBCE = -w_1 * y * \log(p) - w_0 * (1-y) * \log(1-p),$$

where, y is the class indicator (0 or 1), p is the predicted probability for a sentence to belong to class 1 and w_0 and w_1 are the weights for class 0 and class 1 respectively. Here, the value of w_0 and w_1 is the reciprocal of the number of sentences in their respective class.

In Table IV, we present the BiLSTM architecture hyper-parameters and their values that gave best results in our experiment. These were obtained using the nested cross validation with grid search method, as mentioned at the beginning of this section.

TABLE IV. HYPER PARAMETER VALUES FOR BiLSTM ARCHITECTURE

Hyper parameters	Value
Embedding Dimensions	50
Activation Function	Sigmoid
Bi-LSTM Dropout Rate	0.5
Bi-LSTM Recurrent Dropout Rate	0.4
Bi-LSTM Hidden Layers	64
Learning Rate	0.002
Epochs	50
Sentence length	100
Optimizer	Adam

In Table V, we present the results of the classification in terms of precision (Prec), recall (Rec) and F-score (FS) using

each of the four techniques. For our dataset and the parameters chosen, BiLSTM has given the best results. Consider this clause, “Failure by either party to enforce any provision of this agreement will not be deemed a waiver of future enforcement of that or any other provision.” As per our dataset, this is not an obligation. However, this was classified as obligation by SVM, Random Forest and Naïve Bayes since it contained many keywords and combination of keywords that are present in obligation sentences such as “either party”, “waiver of”, “will not be”, “this agreement”, “enforce” etc. However, BiLSTM has correctly classified it as non-obligation by identifying features by utilizing its sequence level information based on attention.

TABLE V. RESULTS OF OBLIGATION EXTRACTION

Techniques	Obligation			Non-obligation		
	Prec	Rec	FS	Prec	Rec	FS
SVM	0.92	0.84	0.88	0.75	0.87	0.81
Random Forest	0.88	0.94	0.91	0.88	0.77	0.83
Naïve Bayes	0.93	0.71	0.8	0.63	0.91	0.74
BiLSTM	0.91	0.94	0.93	0.89	0.84	0.86

B. Classifying Requirements

Since a given obligation sentence can belong to one or more requirement types, we use multilabel classification techniques for their classification. For this, we tried a spectrum of techniques to achieve high accuracy starting with a rule-based approach followed by traditional machine learning based approaches such as Naïve Bayes, Random Forest and SVM, and then finally deep artificial neural network based models including Bidirectional Long Short-Term Memory (BiLSTM) and Bidirectional Encoder Representations from Transformers (BERT). For all the techniques, we have used nested cross validation. In the outer loop, we used grid search method for optimizing hyper parameter values and in the inner loop, we used stratified five-fold cross-validation of the dataset for training a model on a given set of hyper parameter values obtained from the outer loop. We first explain the complexity of our dataset to justify the need to experiment across such a range of techniques. Next, we present details of our experiment and the corresponding results with various techniques.

1) Dataset Complexity

Contracts contain vocabulary, phrases, and linguistic structures that are rarely seen in general natural language corpora [33]. Contractual obligation sentences are usually longer than sentences in news articles, tweets or product reviews that have been most commonly used datasets in information retrieval experiments [33]. Moreover, a single sentence in a contract document can contain a long list of deliverables or activities to be attended to by different stakeholders. This makes the sentence unusually long (100 words or more).

In our dataset, we noticed a significant overlap between feature sets of various requirement types due to certain common topics among these requirement types. For instance, both the requirement types, *Data Privacy* and *Information Security* have some obligations that specify details related to auditing.

As shown in Table I, the distribution of sentences across the requirement types in a contract document is skewed. For instance, the requirement types *Project Delivery* or *Information Security* has higher number of sentences than *Standards* or *Improvement and Innovation*. This creates an imbalance across the classes and makes multilabel classification challenging.

Next, we present details of our experiment and the corresponding results with various techniques.

2) Rule Based Solution

Though we started with this approach for classification, we realized that for practical purposes, this method is not feasible as it is labor intensive, time consuming and therefore is not likely to scale. For a scalable automation, we explored various machine learning and deep learning methods as discussed in subsections 3.B.3, 3.B.4 and 3.B.5.

3) Traditional Machine Learning Techniques

To convert the multilabel classification problem into a one-vs-rest classification problem, we use the binary relevance technique. We train a separate model for each requirement type to identify whether a given sentence belongs or does not belong to that requirement type. Hence, we use 14 different binary classifier models to predict whether a given sentence belongs to one or more requirement types. Since training each model is similar to the binary classification training method, we apply the same techniques in the same manner as presented in subsection 3.A.1.

4) BiLSTM Architecture

To perform the multilabel classification task using BiLSTM architecture, we use the same architecture as described in section 3.A.2. The only change we make for multilabel classification is that we use one sigmoid function for each label in this task; thereby the output layer has 14 nodes, one for each label.

5) BERT Based Solution

In this subsection, we describe BERT implementation to address the problem of multilabel classification of obligation sentences. BERT [7] is a multi-layer deep learning model that utilizes a bidirectional transformer network to create a language model.

BERT gives significantly better results in text processing [7] because, unlike traditional language models that use the previous ‘n’ words to predict the next word, BERT trains a language model that takes the previous and next words together into account when predicting (bi-directional learning). Moreover, it uses attention over a given sequence by using techniques of “Masking” and “Next sentence prediction” to learn the context of each word. BERT also gets an advantage by creating contextual representations (embeddings), not at word level but at the level of wordpieces. For instance, instead of learning word embedding for the word ‘screening’ which appears frequently in sentences of *Screening/Onboarding* requirement type in contracts, BERT creates an embedding for ‘screen’ and for ‘##ing’ and then uses these to recognize different manifestations of “screen” (such as “screened”), if any in test dataset. This helps in dealing with out-of-vocabulary words, i.e., words that were not present in the training dataset but are present in a new input

sentence. This also helps limit the size of vocabulary set of the trained model that improves the efficiency of the model.

As depicted in fig. 3, BERT model is trained in two steps. The first step, called pretraining, creates a language model by executing two tasks known as “Masking” and “Next sentence prediction”. The second step, called fine-tuning, trains a task specific model over the language model created in pretraining step. Next, we explain pretraining and fine-tuning in detail.

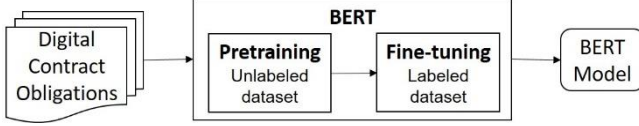


Figure 3. BERT Training Phase

a. Pretraining

The vanilla version of BERT language model has been trained on Wikipedia (2,500 million words) and BooksCorpus (800 million words) [7]. This model needs to be further trained to recognize the domain-specific vocabulary of our contracts dataset. This is called pretraining. For this, we used unlabeled sentences (non-obligation sentences such as information, facts and definitions, i.e., a total of 13,142 sentences or 0.47 million words) from contract documents that are not a part of our labeled dataset. The non-obligation sentences are written in the same style as obligations and contain the same vocabulary and semantic information as the labeled sentences. Therefore, they are useful for pretraining the language model. For better comprehension, we briefly define the hyper parameters in Table VI and we provide the values of various hyper parameters used to optimize the pretraining process in Table VII, as obtained using the grid search approach.

TABLE VI: HYPER PARAMETER DEFINITION

Hyper parameter	Definition
train_batch_size	The number of instances in a single batch that BERT processes
max_seq_length	The maximum length of input sentence in the batch
num_train_steps/ num_train_epochs	The number of training steps needed to create a model that provides best possible results
num_warmup_steps	The number of steps for performing training at a lower learning rate, for better understanding of the training data
learning_rate	The rate at which the model learns the given training data, to create the model
Masked_lm_prob	The percentage of tokens masked out in the input sentence

At the end of pretraining step, we obtain an accuracy of 98.96% for “Masking” and 100% for “Next sentence prediction”. Despite these high accuracy scores, the overall pretraining followed by fine-tuning in BERT is robust to overfitting [37].

TABLE VII: HYPER PARAMETER VALUES

Hyper parameters	Value
train_batch_size	20
max_seq_length	128
num_train_steps	20000

num_warmup_steps	2000
learning_rate	2e-5
masked_lm_prob	0.15

b. Fine-tuning

The labeled dataset consists of sentences in the contract documents that contain information regarding one or more requirement types. First, we convert the contract sentences to BERT input format. Then we use the nested five-fold cross validation technique with grid search to train the BERT model and obtain optimized values of hyper parameters (defined in Table VI). We provide these values in Table VIII.

TABLE VIII: FINE-TUNING HYPER PARAMETER VALUES

Hyper parameters	Value
train_batch_size	20
Max_seq_length	128
Min_train_epochs	25
learning_rate	2e-5

Table IX shows the results in terms of precision (Prec), recall (Rec) and F-score (FS) for classifying the contractual obligations into requirement types using SVM, Random Forest, Naïve Bayes, BiLSTM and BERT. As is evident from Table IX, both machine learning techniques (SVM, Random Forest and Naive Bayes) and deep learning techniques (BiLSTM and BERT) produce high scores for requirement types that have high number of instances in the dataset. BERT produces the best results among them, by a small margin.

However, BERT produces significantly better results (0.2 to 0.6 points higher recall score) among all the techniques where requirement types have lesser number of instances in the dataset, such as in *Personnel Allocation*, *HR Laws*, *Export Laws* and *Standards*. Consider a clause from requirement type *Export Laws*, “Vendor agrees not to disclose, export, re-export or otherwise transfer any confidential information or computer software containing encryption functionality outside the client’s premises or to any other destination or end user outside the country without client’s prior written approval.”

This clause has been misclassified by SVM, Random Forest, Naïve Bayes and BiLSTM as *Information Security*. We observe that this clause contains keywords for *Export Laws* such as “export”, “re-export”, “outside the country” etc., but also keywords for requirement type *Information Security*, such as “confidential information”, “computer software”, “encryption”. Since the machine learning techniques are based on bag-of-words model, they misclassified this clause. Moreover, the sequence of keywords such as “vendor agrees” followed by “disclose” or “transfer” followed by “confidential information” or “vendor agrees” followed by “encryption functionality” indicates that this clause belongs to *Information Security* due to which BiLSTM also misclassifies it. This is because BiLSTM relies on shallow sequential learning. However, BERT correctly classifies it as *Export Laws* since the model understands the writing style of the clauses due to the deep bi-directional learning of its transformer

TABLE IX. RESULTS OF CONTRACT OBLIGATION CLASSIFICATION INTO REQUIREMENT TYPES

Contract Requirement Types	Support Vector Machines			Random Forest			Naïve Bayes			BiLSTM			BERT		
	Prec	Rec	FS	Prec	Rec	FS	Prec	Rec	FS	Prec	Rec	FS	Prec	Rec	FS
Project Delivery	0.91	0.82	0.86	0.83	0.66	0.73	0.89	0.86	0.87	0.88	0.82	0.85	0.86	0.92	0.89
Information Security	0.91	0.80	0.85	0.82	0.69	0.75	0.89	0.88	0.88	0.82	0.89	0.85	0.89	0.90	0.89
Legal Process	0.98	0.55	0.70	0.92	0.40	0.56	0.81	0.78	0.79	0.72	0.83	0.77	0.87	0.84	0.85
Screening/Onboarding	1.00	0.81	0.89	0.94	0.46	0.62	0.86	0.87	0.86	0.89	0.89	0.89	0.91	0.90	0.90
Data Privacy	0.90	0.71	0.79	0.88	0.57	0.69	0.75	0.82	0.78	0.81	0.70	0.75	0.87	0.85	0.86
Vendor Corporate	1.00	0.11	0.20	1.00	0.11	0.20	0.71	0.55	0.62	0.50	0.29	0.37	0.50	0.74	0.60
Improvement and Innovation	1.00	0.17	0.29	1.00	0.16	0.27	0.33	0.25	0.28	1.00	0.75	0.85	1.00	0.61	0.76
Personnel Allocation	0.88	0.34	0.49	0.80	0.17	0.28	0.86	0.56	0.68	0.91	0.58	0.71	0.83	0.82	0.82
HR Client Policy	1.00	0.36	0.53	1.00	0.21	0.34	0.76	0.52	0.62	0.47	0.39	0.43	0.70	0.76	0.73
HR Laws	1.00	0.18	0.30	1.00	0.12	0.21	0.75	0.62	0.68	0.73	0.57	0.64	0.91	0.72	0.80
Third Party IP Licensed	1.00	0.11	0.20	0.32	0.45	0.37	0.46	0.66	0.54	0.47	0.70	0.56	0.71	0.75	0.73
Vendor IP Licensed	0.30	0.17	0.22	0.43	0.37	0.40	0.63	0.65	0.64	0.30	0.12	0.17	0.66	0.70	0.68
Export Laws	1.00	0.50	0.67	1.00	0.16	0.28	0.87	0.58	0.70	0.90	0.69	0.78	1.00	0.92	0.96
Standards	1.00	0.20	0.33	1.00	0.10	0.19	0.50	0.20	0.29	1.00	0.40	0.57	0.71	1.00	0.83

architecture and overcomes the limitation of machine learning techniques and the BiLSTM architecture.

IV. RELATED WORK

We organize our related work into three areas: (1) Business documents (other than Software Requirement Specifications (SRS)) as sources of requirements, (2) Extraction and classification of various artefacts from business contracts and (3) automating the extraction and classifications of artifacts in business documents.

A. Business documents (other SRS) as sources of requirements:

In [30], Breux et al. proposed a technique called semantic parameterization to extract rights and obligations to align requirements with regulations. In [31], the authors analyzed regulatory rules to extract privacy and security requirement. In [26], the authors presented their work on application of machine learning techniques for extracting and classifying non-functional requirements from 11 types of documents out of which one documents is business contracts. Our focus is exclusively on contracts with a specific aim to extract requirements from them.

B. Extraction and classification of various artefacts from business contracts:

Information extraction and classification from documents has been employed extensively for uncovering important details present in complex documents [11]. We found many publications focusing on the extraction of entire contractual sentences or clauses [14, 15, 19] and also extracting of specific contract elements [12, 13, 16, 17, 18]. Gao and Singh [12] proposed a topic modeling based technique to extract business events and their temporal constraints from contract text. Chalkidis and Androutsopoulos [13] employed deep learning methods to extract vari-

ous contract elements such as Title, Party, Start, Effective, Termination, Period, Value, government law, Jurisdiction, Legislative reference and headings. Lippi et al. [14] proposed a machine learning based approach to detect unfair clauses in online contracts. Indukuri et al. [15] proposed a classification approach to extract clause patterns from e-contract documents. They classified contract sentences as clauses or non-clauses and then sub-classified clauses as concerned with payment terms or not. Cutotti and McCreath [16] proposed an approach for classification of components of contract texts using machine learning and hand-coded methods. Gao et al. [17] proposed an approach for discovering service exceptions at the phrase level from a large contract repository. In [18], the authors proposed an approach to automate the disambiguation of contracts. In [19], the authors proposed an automatic model of contract-risk extraction to detect “poisonous” clauses of the contract in order to support contract management for construction companies. None of these studies however focuses on contracts as a source of requirements.

C. Automating the extraction and classifications of artifacts in business documents:

Work related to the application of machine learning and deep learning techniques for automating the extraction and classification of text such as tweets and product reviews can be found in abundance in the literature. However, since our focus is on business documents, we restrict our related work discussion to business documents. In [18], the authors used Naïve Bayes to classify contractual clauses as obligation or non-obligation and extract triggers, actors, timex and action from the extracted obligations using rule based information extraction. In [10], the authors used traditional machine learning algorithm to identify architecturally significant functional requirements (ASFRs) from SRS and further classified them into various ASFR categories. In [21], the authors presented a BiLSTM with Attention based

model for an implementation-centric classification of business rules from SRS. Classification of requirements as ‘requirement’ or ‘information’ using CNN [28], temporal and non-temporal requirement, functional or non-functional categories [29] is also found in the literature. Many publications have focused on automating the classification of non-functional requirements from SRS using TF-IDF [22], multinomial Naïve Bayes classifier coupled with an Expectation Maximization algorithm [23], machine-learning techniques [24], word similarity methods and clustering techniques [25], and SVM with a linear kernel [27]. Our work on extraction and classification of requirements from contracts employs a whole range of techniques both traditional and state-of-the-art such as BERT.

V. VALIDATION STUDY

To evaluate the usefulness of the approach, we used a new contracts document (250 pages, 1608 sentences) of an ongoing project from finance domain. We shared the requirement types and their definitions with two volunteers (who did not participate in the exploratory study), one, a Requirements Engineer and the other from the contracts governance team in the organization. We asked the volunteers to manually annotate the sentences as obligations or non-obligations and further the obligations sentences as one or more requirement types identified in our exploratory study. The Requirements Engineer was asked to identify architectural requirements while the governance team member was asked to identify governance requirements. This activity took 35 person hours. Both the experts were able to correctly identify all the corresponding requirement types. For comparison, we obtained the extraction and classification output using the automated method described in this paper. The proposed automated method took 3 minutes to generate the output with F-score of 85.8 %. While the manually annotated output is 100% accurate, the time and effort needed for manual annotation is considerably high as compared to the automation. They expressed that making architectural requirements explicit before the RE exercise begins can equip Requirements Engineers to ask the right set of questions to the customers during requirements elicitation. Also, they pointed out that, when a large number of contracts are to be processed manually, the accuracies could suffer as a result of fatigue setting in.

VI. THREATS TO VALIDITY

A first possible threat to validity is the comprehensiveness of requirement types identified in this study. We cannot claim that the requirement types identified by us are universal and present in every contracts document. However, it is reasonable to expect similar set of requirement types in any set of SE contracts. This claim is substantiated by our validation study reported in Section V.

The second threat to validity is the extent to which the coding words of the participants in the exploratory study can be considered the best representation for the requirement types. In our study, we have participants from SE domain, governance team, and legal domain. This ensures that the knowledge of all the three domains is used in vetting the coding words and ensuring

that the requirement types identified through this study reasonably represent requirements likely to be present in contracts.

The third threat is our dataset – Are the contracts in our dataset a good representation of SE contracts? Since our contracts dataset consists of 20 contracts from SE projects in 9 different domains, we expect our dataset to be a good representation of SE contracts. Legal experts from customers’ end are equal partners involved in drawing up contracts. Therefore, the contracts represent a multitude of domain-specific variants adequately as vetted by experts from at least 20 different customers’ organizations. These factors contribute to the generalizability of the dataset. Additionally, the contracts language is significantly repetitive [36]. As a result, we expect contracts to be linguistically similar across organization, and therefore, our findings, mostly generalizable. We used a large set of real-world contracts (20 contracts) comprising of over 18,000 sentences. Although this cannot serve for forming universal claims, it injects realism [35] in our experiments.

VII. CONCLUSION

Since contracts aim to accurately outline all mandatory requirements in the form of obligations, we felt they could be the starting point of an RE exercise. We determined the types of requirements that can be gleaned from contracts through an exploratory study. We automated the extraction and classification of requirements and attained high accuracy of automation through experiments employing a range of traditional and recent techniques such as BERT.

The architectural requirements form 39.73% of the requirements we extracted from the contractual obligations. This means a sizeable proportion of obligations contain SE-specific requirements knowledge and contracts are therefore a credible source of SE requirements.

The remaining 60.27% are governance-specific. While a lot of RE research exists on identification and detailing of different kinds of requirements (such as functional, non-functional, and architectural), there seems to be hardly any work on explicitly identifying governance requirements for SE projects. Our findings lead us to believe that governance requirements need to find their legitimate place on the RE landscape; they should not be left out of scope of requirements elicitation exercises.

We therefore posit that RE as a discipline has an even wider scope than SE of which it is traditionally considered to be a sub-discipline. We appeal to the RE community to examine this stance and include governance requirements within the scope of RE. This will stimulate interesting and useful discussions on exploring different sources of such requirements and methods for their identification, extraction, and classification. The upcoming International Conference on Software Engineering (ICSE 2020) has its very first workshop on SE governance (<https://www.es.mdh.se/GISE/>) which underlines our position that governance requirements need to be designated as a distinct type of requirements in RE itself so that they are not left out of scope during elicitation.

REFERENCES

- [1] Hochreiter, Sepp, and Jürgen Schmidhuber. "Long short-term memory." *Neural computation* 9.8 (1997): 1735-1780.
- [2] Coale, K., (1992) British Petroleum Outsources I.S., *InforWorld*, 14, 5, 47.
- [3] Ian Sommerville and Pete Sawyer, *Requirements Engineering: A Good Practice Guide* (Wiley, 1997).
- [4] <https://www.infosysbpm.com/offering/functions/legal-process-outsourcing/white-papers/Documents/contract-process-helping-hurting.pdf> last accessed on 31-07-2019
- [5] Curtotti, M. and McCreath, E., 2010. *Corpus Based Classification of Text in Australian Contracts*, In *Proceedings of Australasian Language Technology Association Workshop* (pp. 18–26).
- [6] <https://www.contractworks.com/blog/why-use-e-signatures> Last accessed on 22-08-2019
- [7] Devlin J., Chang M., Lee K., Bert: pre-training of deep bidirectional transformers for language understanding. In: *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, Volume 1 (Long and Short Papers), Minneapolis, MN, USA. pp. 4171–4186.
- [8] Mikolov, T., Chen, K., Corrado, G., & Dean, J. (2013). Efficient estimation of word representations in vector space. *arXiv preprint arXiv:1301.3781*.
- [9] Goldberg, Y., & Levy, O. (2014). word2vec Explained: deriving Mikolov et al.'s negative-sampling word-embedding method. *arXiv preprint arXiv:1402.3722*
- [10] Anish, P. R., Balasubramaniam, B., Sainani, A., Cleland-Huang, J., Daneva, M., Wieringa, R. J., & Ghaisas, S. (2016, May). Probing for requirements knowledge to stimulate architectural thinking. In *Proceedings of the 38th International Conference on Software Engineering* (pp. 843-854).
- [11] G. K. Palshikar, Techniques for named entity recognition: a survey, in S. Bruggemann, C. D'Amato (Ed.s), *Collaboration and the Semantic Web: Social Networks, Knowledge Networks and Knowledge Resources*, IGI Global, 2012, pp. 191–217
- [12] X. Gao, M.P. Singh, Mining contracts for business events and temporal constraints in service engagements, *IEEE Transactions on Services Computing*, 2014, 7(3), pp.427-439
- [13] I. Chalkidis, I. Androutsopoulos, A Deep Learning Approach to Contract Element Extraction, In *JURIX*, 2017, pp. 155-164
- [14] Lippi, M., Palka, P., Contissa, G., Lagioia, F., Micklitz, H.W., Panagis, Y., Sartor, G. and Torroni, P., 2017. Automated Detection of Unfair Clauses in Online Consumer Contracts. In *JURIX* (pp. 145-154)
- [15] K. V. Indukuri and P. Radha Krishna, Mining e-contract documents to classify clauses, in *Proc. of the 3rd Annual ACM Bangalore Conference (Compute)*. Bangalore, India: ACM, 2010, pp. 1–5.
- [16] M. Curtotti, E. McCreath, Corpus based classification of text in Australian contracts, In *Proc. of the Australasian Language Technology Association Workshop*, pp. 18–26, Melbourne, Australia, 2010
- [17] Gao, X., Singh, M.P. and Mehra, P., 2011. Mining business contracts for service exceptions. *IEEE Transactions on Services Computing*, 5(3), pp.333-344.
- [18] Anish, P.R., Sainani, A., Ramrakhiyani, N., Pawar, S., Palshikar, G. and Ghaisas, S., 2019. Towards Disambiguating Contracts for their Successful Execution-A Case from Finance Domain. In *Proceedings of the First Workshop on Financial Technology and Natural Language Processing* (pp. 8-13).
- [19] Lee, J., Yi, J.S. and Son, J., 2019. Development of automatic-extraction model of poisonous clauses in international construction contracts using rule-based NLP. *Journal of Computing in Civil Engineering*, 33(3), p.04019003.
- [20] P. R. Anish, M. Daneva, J. Cleland-Huang, R. J. Wieringa and S. Ghaisas, "What you ask is what you get: Understanding architecturally significant functional requirements," *2015 IEEE 23rd International Requirements Engineering Conference (RE)*, Ottawa, ON, 2015, pp. 86-95.
- [21] Anish, P.R., Sainani, A., Ahmed, A. and Ghaisas, S., 2019, September. Implementation-Centric Classification of Business Rules from Documents. In *2019 IEEE 27th International Requirements Engineering Conference Workshops (REW)*(pp. 227-233). IEEE.
- [22] J. C.-Huang, R. Settini, X. Zou, P. Solc, "Automated classification of non-functional requirements", *Proceedings of IEEE International Requirements Engineering conference*, vol.12, pp.103-120, May 2007
- [23] Casamayor, A., Godoy, D. and Campo, M., 2010. Identification of non-functional requirements in textual specifications: A semisupervised learning approach. *Information and Software Technology*, 52(4), pp.436-445
- [24] Slinkas, J. and Williams, L., 2013, May. Automated extraction of non-functional requirements in available documentation. In *2013 1st International Workshop on Natural Language Analysis in Software Engineering (NaturaLiSE)* (pp. 9-16). IEEE
- [25] Mahmoud, A. and Williams, G., 2016. Detecting, classifying, and tracing non-functional software requirements. *Requirements Engineering*, 21(3), pp.357-381
- [26] Slinkas, J. and Williams, L., 2013, May. Automated extraction of non-functional requirements in available documentation. In *2013 1st International Workshop on Natural Language Analysis in Software Engineering (NaturaLiSE)* (pp. 9-16). IEEE.
- [27] W. Zhang, Y. Yang, Q. Wang, and F. Shu, "An Empirical Study on Classification of Non-Functional Requirements," in *The Twenty-Third International Conference on Software Engineering and Knowledge Engineering (SEKE 2011)*, 2011, pp. 190–195
- [28] Winkler, J. and Vogelsang, A., 2016, September. Automatic classification of requirements based on convolutional neural networks. In *2016 IEEE 24th International Requirements Engineering Conference Workshops (REW)* (pp. 39-45). IEEE.
- [29] J. H. Hayes, W. Li, and M. Rahimi, "Weka meets TraceLab: Toward Convenient Classification: Machine Learning for Requirements Engineering Problems: A Position Paper," in *1st IEEE International Workshop on Artificial Intelligence for Requirements Engineering*, ser. AIRE, 2014, pp. 9–12
- [30] Breaux, T.D., Vail, M.W. and Anton, A.I., 2006, September. Towards regulatory compliance: Extracting rights and obligations to align requirements with regulations. In *14th IEEE International Requirements Engineering Conference (RE'06)* (pp. 49-58). IEEE.
- [31] Breaux, T. and Antón, A., 2008. Analyzing regulatory rules for privacy and security requirements. *IEEE transactions on software engineering*, 34(1), pp.5-20.
- [32] C. Charmaz, *Constructing Grounded Theory: A Practical Guide through Qualitative Analysis*, Sage, Thousand Oaks, 2006.
- [33] Zhang, R., Yang, W., Lin, L., Tu, Z., Xie, Y., Fu, Z., Xie, Y., Tan, L., Xiong, K. and Lin, J., 2020. Rapid Adaptation of BERT for Information Extraction on Domain-Specific Business Documents. *arXiv preprint arXiv:2002.01861*.
- [34] Egelund-Müller, B., Elsmann, M., Henglein, F. and Ross, O., 2017. Automated execution of financial contracts on blockchains. *Business & Information Systems Engineering*, 59(6), pp.457-467.
- [35] Wieringa, R.J. and Daneva, M. (2015). Six strategies for generalizing software engineering theories. *Science of computer programming*, 101(April 2015), 136-152.
- [36] Simonson, D., Broderick, D., & Herr, J. (2019, June). The Extent of Repetition in Contract Language. In *Proceedings of the Natural Legal Language Processing Workshop 2019* (pp. 21-30).
- [37] Hao, Y., Dong, L., Wei, F., & Xu, K. (2019). Visualizing and Understanding the Effectiveness of BERT. *arXiv preprint arXiv:1908.05620*.
- [38] Pedregosa, Fabian, et al. "Scikit-learn: Machine learning in Python." *the Journal of machine Learning research* 12 (2011): 2825-2830.
- [39] Chollet, François. "Keras: The python deep learning library." *Astrophysics Source Code Library* (2018).